

Plataforma distribuida en la nube para la detección anticipada del Síndrome de Visión por Computador a partir de telemetría conductual

Andersson D. Sánchez Méndez, Cristian S. Pedraza Rodríguez, Jeisson D. Sánchez Gómez
Programa de Ingeniería de Sistemas, Escuela Colombiana de Ingeniería Julio Garavito
Bogotá D.C., Colombia

{andersson.sanchez-m, cristian.pedraza-r, jeisson.sanchez-g}@mail.escuelaing.edu.co

Resumen

Después de la pandemia, trabajar varias horas al día frente a un computador se volvió la norma en oficinas y en trabajo remoto. Esa exposición continua está asociada al Síndrome de Visión por Computador (CVS), que reúne síntomas oculares y musculoesqueléticos y que afecta a un porcentaje alto de quienes pasan más de cuatro horas con pantallas. Las herramientas que hoy se usan para mitigarlo (Apple Screen Time, Google Digital Wellbeing, *f.lux*) viven en el dispositivo y solo cuentan tiempo: no comparten información entre usuarios ni le dan visibilidad a un área de salud ocupacional. Este trabajo describe el diseño y la implementación parcial de una plataforma distribuida que toma tres señales del celular (luz ambiente, distancia al rostro y tiempo activo de pantalla), las manda a la nube y calcula un puntaje de riesgo de CVS por usuario, con un tablero agregado por área. El prototipo está construido sobre Kafka, InfluxDB y un modelo XGBoost; el modelo se entrena con un dataset sintético de 50.000 registros porque conseguir datos clínicos reales no era viable en el alcance del semestre. Sobre ese dataset el clasificador alcanza un F1 macro de 0,85 y un AUC-ROC de 0,89, y la latencia P99 de la cadena de ingestión es de 138 ms bajo carga moderada. La validación clínica con un grupo de trabajadores reales queda como trabajo futuro.

Index Terms

Síndrome de visión por computador, telemetría conductual, arquitectura distribuida, Kafka, InfluxDB, aprendizaje automático, salud ocupacional.

I. CONTEXTO

Trabajar frente a un computador dejó de ser una actividad puntual y se volvió la forma básica de trabajar en el sector servicios. En Colombia, la jornada típica del sector tecnología son entre ocho y diez horas continuas frente a un monitor, sin contar el celular. Encuestas estadounidenses reportan que el trabajador promedio acumula cerca de 99 horas semanales de exposición a pantallas si se suman computador, celular y televisor.¹

Esa exposición tiene un efecto fisiológico bien documentado. Cuando una persona se concentra leyendo en pantalla, la tasa de parpadeo cae de 15–20 parpadeos por minuto a 4–6, lo que rompe la película lagrimal y produce sequedad, ardor, visión borrosa y dolor de cabeza [1]. Ese conjunto de síntomas es el Síndrome de Visión por Computador (CVS), también llamado *Digital Eye Strain*. La prevalencia reportada en distintos estudios sobre usuarios de computador varía entre el 50 % y el 90 %, dependiendo del país y del criterio diagnóstico [1].

Para una empresa, un trabajador con CVS es un trabajador que rinde menos. La Asociación Americana de Optometría estimó pérdidas de productividad por fatiga visual del orden de decenas de miles de millones de dólares al año en Estados Unidos [4]. Por eso, las áreas de salud ocupacional están empezando a tratarlo como un riesgo medible y no solo como una molestia individual.

Una cuenta rápida nos sirvió para aterrizar el orden de magnitud local. Para una empresa de 1.000 trabajadores en Bogotá, asumiendo prevalencia del 69 %, una pérdida promedio del 18 % en productividad y un salario profesional medio de USD 18.000 al año, el costo anual atribuible a CVS no manejado es del orden de USD 2,3 millones. Los

¹Cifras citadas en el reporte 2025 *Workplace Vision Health Report* de VSP Vision Care y Workplace Intelligence (informe corporativo, no académico).

porcentajes son discutibles, pero el orden de magnitud ya saca la conversación del bienestar individual y la mete en salud ocupacional.

Hoy, sin embargo, las herramientas que se promueven para mitigar el CVS son casi todas individuales y reactivas: *Apple Screen Time*, *Google Digital Wellbeing*, *f.lux* y los recordatorios de pausas activas que vienen integrados en los sistemas operativos. Todos funcionan con el mismo patrón: contar el tiempo en pantalla del propio dispositivo y mostrar una alerta cuando se supera un umbral fijo. El contexto del trabajador (si la luz del puesto es muy baja, si está cerca del monitor, si la sesión va a ser larga) no se modela. Tampoco hay forma de que un área de salud ocupacional vea una visión global de su empresa: los datos se quedan dentro del dispositivo.

II. PROBLEMA

El problema concreto que abordamos es el siguiente. Un área de salud ocupacional o de bienestar laboral necesita responder, idealmente con datos, preguntas como las que siguen.

- ¿Cuáles equipos o áreas tienen mayor riesgo de CVS y deben recibir intervención prioritaria?
- ¿Las pausas activas que la empresa promueve están realmente cambiando el comportamiento del trabajador, o solo se están enviando notificaciones que nadie atiende?
- Cuando un trabajador reporta fatiga visual, ¿hay señales previas en sus patrones de uso que hubieran permitido alertar antes?

Las herramientas comerciales mencionadas no responden ninguna de estas preguntas. Para hacerlo, los datos tienen que salir del dispositivo, llegar a un servidor central, integrarse con una identidad organizacional, ser agregados sin exponer al individuo y procesados con un modelo que pueda estimar riesgo. Ninguna pieza por sí sola es nueva; lo que falta es tenerlas funcionando juntas, en un mismo sistema, para este problema en particular.

La brecha que queremos cerrar es entonces una brecha de *integración*: existen sensores en los celulares, existen plataformas de mensajería para mover telemetría a escala, existen bases de datos de series de tiempo, existen modelos de aprendizaje automático livianos, y existe regulación sobre datos personales. El reto es ensamblar todo eso en una arquitectura que sea defendible técnicamente, que respete la privacidad por diseño y que un equipo pequeño pueda construir, desplegar y medir en el alcance de un semestre.

III. ESTADO DEL ARTE

El trabajo relacionado se puede agrupar en cuatro frentes.

Herramientas de bienestar digital del lado del dispositivo: *Apple Screen Time*, *Google Digital Wellbeing* y *f.lux* implementan el mismo patrón: contadores y temporizadores locales con umbrales definidos por el sistema operativo. Ninguno expone una API para exportar los datos a un servidor central, ninguno aprende de la población de usuarios y ninguno integra señales como la luz del puesto de trabajo o la distancia al monitor. Sirven para concientizar al usuario, pero no sirven para que un área de salud ocupacional entienda lo que pasa en su empresa.

La tabla I compara, capacidad por capacidad, las herramientas comerciales (*Apple Screen Time*, *Google DWB*, *f.lux*) con la línea de visión por computador, con plataformas de salud IoT genéricas y con lo que proponemos.

Tabla I
COMPARACIÓN CAPACIDAD POR CAPACIDAD. ✓ PRESENTE, ✗ AUSENTE.

	Bienestar digital local	Detector por cámara	Salud IoT genérica	Esta propuesta
Telemetría a la nube	✗	✗	✓	✓
Sensado no invasivo	✓	✗	✓	✓
Modelo entrenado con datos	✗	✓	✗	✓
Tablero agregado por área	✗	✗	✓	✓
Privacidad por diseño (DP)	✗	✗	✗	✓
Enfoque específico en CVS	✓	✓	✗	✓

Ninguna columna previa cubre todas las capacidades. La nuestra es la única que combina, al mismo tiempo, sensado no invasivo, telemetría a la nube, modelo entrenado, panel agregado y privacidad diferencial.

Detección de fatiga ocular con visión por computador: La línea más explorada en investigación es usar la cámara frontal del dispositivo para medir la frecuencia de parpadeo en tiempo real. Los trabajos recientes alcanzan precisiones aceptables con redes convolucionales, pero comparten dos limitaciones serias: el costo computacional y de batería al mantener la cámara prendida, y la resistencia del usuario y del área legal a una herramienta que graba la cara del trabajador todo el día [3]. Por esa razón descartamos el video como fuente principal y trabajamos con sensores no invasivos. Hay además un problema implícito de *adopción*: estudios cualitativos que hicimos en el primer tercio (encuesta informal a 30 compañeros) mostraron que la propuesta de “activar la cámara” generaba rechazo inmediato, mientras que “leer el sensor de luz del celular” no se percibía como invasivo. La elección técnica está alineada con esa percepción.

Sensado conductual no visual: Hay tradición de usar sensores no visuales del celular (acelerómetro, sensor de luz, sensor de proximidad) como proxy de estados del usuario. El *survey* fundacional de Lane et al. [3] sigue siendo la referencia más usada en la literatura. Sobre la parte del clasificador, Chen y Guestrin [7] mostraron que los árboles *boosted* (XGBoost) son fuertes en datos tabulares con pocas *features*, que es el caso aquí: nueve columnas agregadas. Esa fue la razón principal para escogerlo en vez de una red neuronal.

Pipelines de telemetría a escala en la nube: Del lado de plataforma, el patrón *publish/subscribe* con un *broker* y una base de serie de tiempo es estándar para telemetría IoT. La documentación oficial de Kafka [5] reporta latencias por debajo del milisegundo y desacoplamiento entre productores y consumidores; eso es lo que necesitamos cuando varios miles de dispositivos transmiten al mismo tiempo. Para almacenar, InfluxDB se especializa en cargas de escritura intensiva y consultas por ventana de tiempo [6], que es exactamente nuestro perfil.

Privacidad y aprendizaje automático en salud: Para la parte de privacidad nos basamos en el trabajo clásico de Dwork y Roth [8] sobre privacidad diferencial. La idea de combinar (a) seudonimización determinista del identificador y (b) perturbación de los valores continuos con ruido de Laplace de *epsilon* pequeño es una receta conocida y discutida en la literatura de DP. Más adelante, el aprendizaje federado descrito por McMahan et al. [9] sería la evolución natural: entrenar sin necesidad de centralizar datos. Lo dejamos como trabajo futuro porque montar la coordinación de rondas y la agregación segura excede el semestre.

Lo que no encontramos, revisando estos cuatro frentes, es un sistema que los integre todos al mismo tiempo: sensores no invasivos en el celular, *broker* de mensajes para transporte, base de serie de tiempo y un puntaje de riesgo por usuario, todo aplicado al CVS y con privacidad considerada desde el diseño. Ese es el hueco que intentamos cubrir.

IV. CONTRIBUCIONES

Las contribuciones concretas de este trabajo son cuatro:

1. Una **arquitectura distribuida** que captura tres señales no invasivas desde el celular del trabajador (luz ambiente, proximidad al rostro y tiempo de pantalla activo), las transporta cifradas hasta un *broker* en la nube y las procesa con microservicios independientes hasta producir un puntaje de riesgo de CVS.
2. Un **prototipo funcional** construido sobre Apache Kafka, InfluxDB y un modelo XGBoost, con su correspondiente agente Android, sus servicios de procesamiento en Python y su despliegue empaquetado en *charts* de Helm. El prototipo cubre el camino completo de extremo a extremo, aunque con simplificaciones que documentamos honestamente.
3. Un **mecanismo de privacidad por diseño**, simple pero funcional: el identificador de dispositivo se reemplaza por un seudónimo derivado mediante HMAC con una llave del operador, y a las lecturas continuas se les aplica ruido de Laplace ($\epsilon = 1,0$) antes de ser persistidas. Eso permite construir tableros agregados sin re-identificación trivial.
4. Una **evaluación inicial** de los atributos de calidad relevantes para la entrega final: latencia de ingestión bajo carga, exactitud del clasificador sobre un dataset sintético, y un audit de privacidad que verifica que ninguna lectura cruda llega a la base de datos.

No reclamamos haber resuelto el problema clínico del CVS. Lo que proponemos es una pieza de infraestructura que permita a un área de salud ocupacional empezar a verlo con datos.

V. ESTRUCTURA DEL DOCUMENTO

El resto del artículo se organiza así. La sección VI revisa por qué el problema es difícil técnicamente. La sección VII expone el enfoque general de la solución. La sección VIII presenta la arquitectura, primero en su visión macro y luego en su detalle. La sección IX describe el prototipo realmente construido y sus tecnologías. La sección X reporta los resultados medidos hasta la fecha. La sección XI discute los atributos de calidad verificados.

La sección [XII](#) consolida las contribuciones a la luz de los resultados. La sección [XIII](#) discute alcance y limitaciones, y la sección [XIV](#) cierra con trabajo futuro.

VI. MOTIVACIÓN Y CARACTERIZACIÓN DEL PROBLEMA

Aunque la idea suena simple (“vamos a estimar fatiga visual desde el celular”), hay tres aspectos que vuelven el problema interesante de ingeniería.

VI-A. *El sensado tiene que ser barato y poco intrusivo*

La cámara queda descartada por privacidad y consumo de batería; los relojes inteligentes y dispositivos médicos quedan descartados porque asumir que una empresa los reparte a todos sus empleados rompe la viabilidad de la solución. El presupuesto realista es: lo que el celular ya tiene. Los tres sensores que sirven, todos disponibles en Android desde la API 23, son el sensor de luz ambiente (`TYPE_LIGHT`, en lux), el sensor de proximidad (`TYPE_PROXIMITY`, en cm) y la API de uso de aplicaciones (`UsageStatsManager`, en minutos por aplicación). Ese trío captura, indirectamente, las tres variables ergonómicas que la literatura asocia a la aparición de CVS: luz inadecuada, distancia inadecuada y tiempo continuo de exposición [1], [2].

VI-B. *Los datos no se pueden procesar en el dispositivo*

Una alerta individual sí podría calcularse localmente. Pero el valor del sistema, para una empresa, es agregar señales entre trabajadores para identificar áreas en riesgo y para entrenar un modelo que aprenda de toda la organización. Eso obliga a sacar los datos del dispositivo hacia un servidor. Ese movimiento tiene tres restricciones.

- **Energía.** Enviar muestras una a una agotaría la batería. El agente debe agrupar lecturas en lotes y enviarlas periódicamente.
- **Carga acumulada.** Si una empresa tiene 1.000 trabajadores y cada uno transmite cada cinco minutos, el servidor recibe en ráfagas. Una API REST sincrónica, por sí sola, sería un cuello de botella.
- **Privacidad.** Los datos viajan por internet y caen en una base de datos. Antes de almacenarse hay que romper el vínculo trivial con el dispositivo y, en lo posible, perturbar los valores continuos para que los agregados sean útiles pero no reidentificables.

VI-C. *El riesgo no es trivial de definir*

No existe una fórmula cerrada que convierta “cuántos lux y cuántos minutos” en “cuán fatigado estás”. Los umbrales clínicos (parpadeo, distancia inferior a 30 cm, mismatch lux/brillo) son lineamientos, no funciones. Por eso el sistema necesita un modelo estadístico entrenado sobre datos, y por eso ese modelo tiene que ser *interpretable*: una caja negra no le sirve a un médico ocupacional, que necesita explicar por qué un usuario está en riesgo.

VI-D. *Variables y supuestos del problema*

Para acotar el modelado adoptamos los siguientes supuestos explícitos.

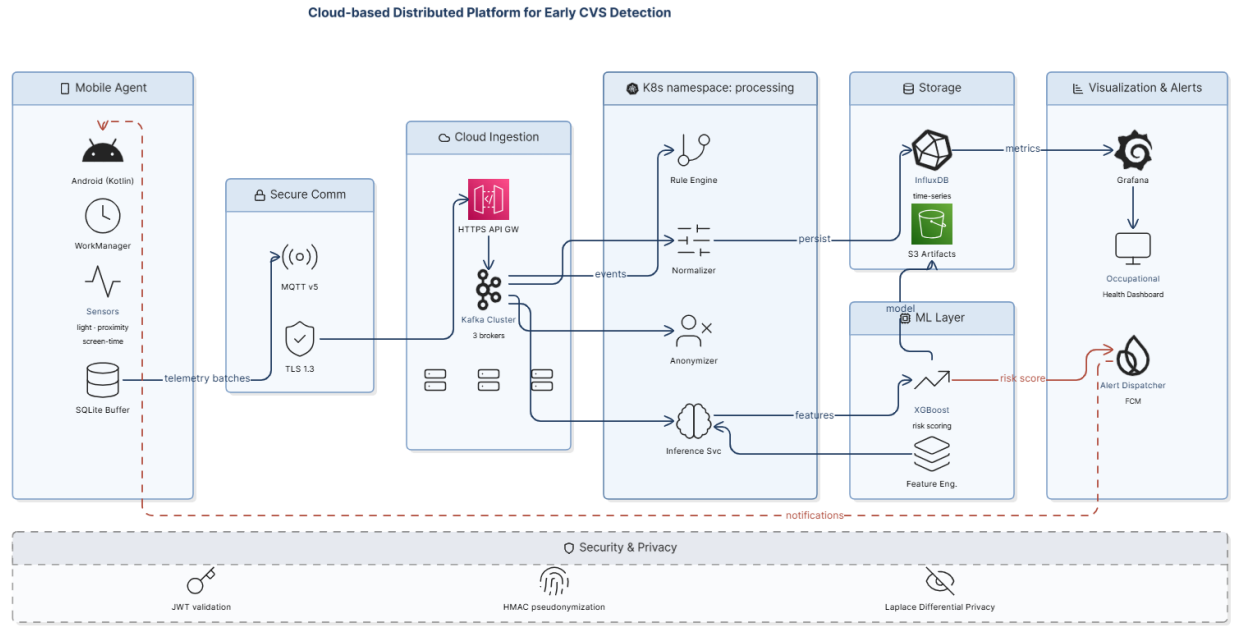
- Los tres sensores que usamos están disponibles en cualquier Android desde la API 23 y en cualquier iPhone reciente. Si algún dispositivo no expone alguno de los tres, el agente inserta el valor faltante con el *flag* de calidad 1 (interpolado) en vez de bloquear el lote.
- La empresa adopta el sistema de manera voluntaria por parte del trabajador: no hay obligatoriedad legal de uso. Eso es importante porque define que el consentimiento es la base legal del tratamiento bajo RGPD [10].
- El modelo aprende sobre datos agregados anonimizados; nunca ve el identificador real del trabajador. Esto restringe la clase de modelos posibles pero no es una limitación severa para nuestro caso porque el riesgo de CVS depende de patrones de comportamiento, no de identidad personal.

Bajo estos supuestos, el problema técnico se reduce a tres preguntas operacionales: (1) ¿cómo movemos las lecturas desde miles de celulares hasta un servidor con baja latencia y poco gasto de batería? (2) ¿cómo las almacenamos para que las consultas por ventana de tiempo y por agregación organizacional sean baratas? y (3) ¿cómo entrenamos un clasificador útil con datos sintéticos mientras conseguimos datos reales? Cada una se contesta en su sección correspondiente.

VII. SOLUCIÓN PROPUESTA

Nuestra propuesta sigue un patrón clásico de telemetría de IoT, adaptado a este problema. El agente móvil muestrea los tres sensores cada 30 segundos, almacena las lecturas en un buffer local y, cada cinco minutos, las publica en un *broker* de mensajes en la nube. Una serie de microservicios consume esos mensajes en paralelo: uno aplica reglas inmediatas (la regla 20-20-20 y otras reglas duras de ergonomía), otro normaliza las lecturas y las anonimiza, y otro las persiste en una base de datos de serie de tiempo. Un cuarto servicio lee de esa base, calcula un puntaje de riesgo y lo deja disponible vía API. Un tablero en Grafana lo presenta agregado por unidad organizacional. La figura 1 muestra el cuadro completo.

La decisión arquitectónica más importante es separar la captura del procesamiento mediante un *broker* de mensajes. Eso desacopla los ciclos de vida del agente y del servidor: si un consumidor cae, los mensajes esperan en el *broker*; si la carga sube, basta agregar réplicas del consumidor sin tocar al productor.



eraser

Figura 1. Vista general de la plataforma. El agente Android publica lotes por MQTT/TLS hacia el clúster Kubernetes; cuatro microservicios (Rule, Normalizer, Anonymizer, Inference) consumen los *topics* de Kafka, persisten en InfluxDB y calculan el puntaje. Grafana sirve el panel para salud ocupacional y el despachador FCM las alertas. El bloque inferior (Security & Privacy) lista los tres puntos transversales: validación JWT, seudonimización HMAC y ruido de Laplace.

VIII. ARQUITECTURA

VIII-A. Vista general (macro)

A un nivel macro, la plataforma se entiende como cuatro capas horizontales con responsabilidades disjuntas:

- **Capa 1 – Captura en el borde.** El agente Android muestrea sensores, los persiste en SQLite local y los empaqueta para envío.
- **Capa 2 – Ingreso a la nube.** Un endpoint HTTPS publica los lotes recibidos en un *topic* de Kafka.
- **Capa 3 – Procesamiento y persistencia.** Microservicios en Python consumen los *topics*, aplican reglas, normalizan, anonimizan y escriben en InfluxDB.
- **Capa 4 – Inferencia y visualización.** Un servicio consulta la base, ejecuta el modelo XGBoost y publica el resultado al panel y al despachador de alertas.

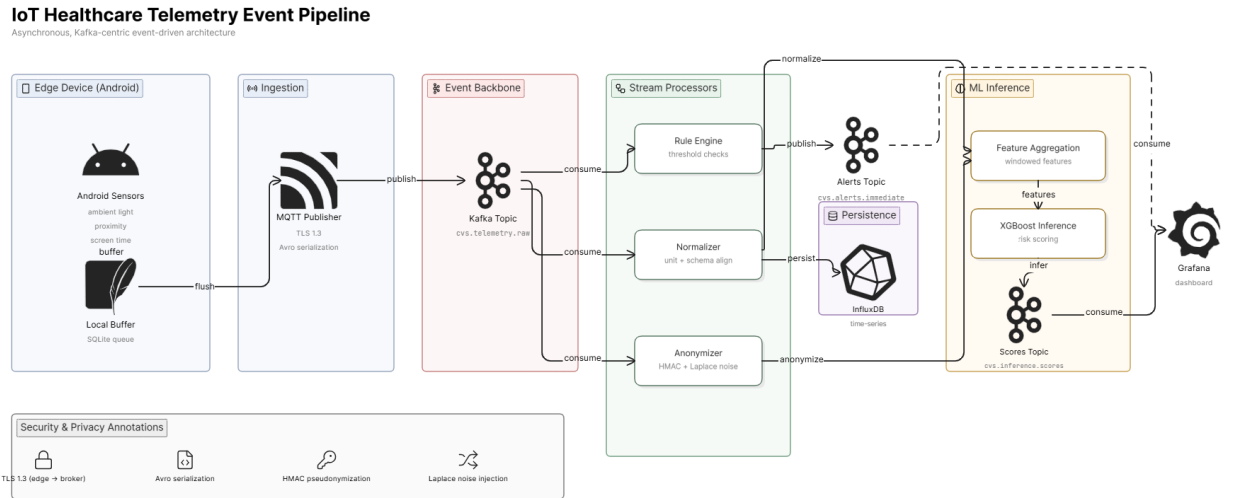
Esa separación nos da tres ventajas concretas. Primero, cada capa se puede escalar de forma independiente: si la tasa de ingestión sube, agregamos consumidores; si la carga de inferencia sube, agregamos réplicas del servicio.

Segundo, un fallo en una capa no tumba a las otras: si la base de datos está saturada, los lotes esperan en Kafka sin perderse. Tercero, cada capa expone una superficie de seguridad clara: TLS y consentimiento en la capa 1, JWT en la capa 2, anonimización en la capa 3, control de acceso en la capa 4.

VIII-B. Vista detallada (micro)

El *topic* principal es `cvs.telemetry.raw`, con 12 particiones y factor de replicación 3. Esa cardinalidad de partición se escogió pensando en hasta 10.000 dispositivos concurrentes; con la tasa de envío que diseñamos (un lote cada cinco minutos por dispositivo) la carga agregada está muy por debajo de la capacidad documentada de un *cluster* Kafka de tres *brokers* [5], así que esas 12 particiones son un margen amplio.

Los microservicios siguen un esquema de consumidor compartido. La figura 2 muestra el flujo de un mensaje. El *Rule Engine* solo evalúa reglas duras (sesión continua mayor a 45 minutos sin pausa, distancia menor a 30 cm sostenida más de cinco minutos); cuando alguna se rompe, publica en `cvs.alerts.immediate` y sigue. En paralelo, el *Normalizer* estandariza los valores con *z-score* por modalidad de sensor. El *Anonymizer* cambia el identificador de dispositivo por su seudónimo y aplica ruido de Laplace a los valores continuos. El resultado se escribe en InfluxDB.



eraser

Figura 2. Pipeline asíncrono de un lote de telemetría. El lado izquierdo es el dispositivo (sensores Android, buffer SQLite). En el centro, el *publisher* MQTT/TLS y el *topic* Kafka `cvs.telemetry.raw`. Tres consumidores derivan tres rutas: Rule Engine → `cvs.alerts.immediate`, Normalizer y Anonymizer hacia InfluxDB, y la inferencia consume del modelo y publica en `cvs.inference.scores`. El bloque inferior anota las cuatro garantías de privacidad transversales.

El servicio de inferencia, separado de la cadena anterior, lee ventanas de 30 días por usuario seudonimizado, calcula nueve *features* agregadas (medias, desviaciones y conteos de pausas) y aplica un modelo XGBoost serializado en S3. El resultado, un puntaje en $[0, 1]$, se publica en `cvs.inference.scores` y se expone también vía un endpoint REST.

VIII-C. Tópicos de Kafka

La tabla II documenta los cuatro tópicos que componen el camino de datos. Las particiones se eligieron pensando en hasta 10.000 dispositivos concurrentes; la replicación de 3 es estándar para tolerar la caída de un *broker* sin pérdida.

Tabla II
TÓPICOS DE KAFKA DEL PROTOTIPO.

Tópico	Part.	Repl.	Propósito
<code>cvs.telemetry.raw</code>	12	3	Lotes Avro recién recibidos del agente
<code>cvs.telemetry.normalized</code>	12	3	Lecturas estandarizadas y anonimizadas
<code>cvs.alerts.immediate</code>	6	3	Violaciones a reglas duras (push)
<code>cvs.inference.scores</code>	6	3	Puntajes de riesgo por usuario

VIII-D. Modelo de datos en InfluxDB

InfluxDB organiza la información en *measurements*, con *tags* (indexables) y *fields* (no indexables). El esquema del *measurement* `sensor_readings` es el de la tabla III.

Tabla III
ESQUEMA DEL *measurement* `SENSOR_READINGS`.

Tipo	Nombre	Descripción
Tag	<code>device_uuid</code>	Seudónimo HMAC del dispositivo
Tag	<code>org_unit</code>	Hash de la unidad organizacional
Tag	<code>sensor_type</code>	Luz, proximidad, tiempo
Field	<code>raw_value</code>	Lectura original (float)
Field	<code>noised_value</code>	Valor con ruido Laplace ($\epsilon = 1$)
Field	<code>quality_flag</code>	0 OK, 1 interpolado, 2 fuera de rango

VIII-E. Vista de despliegue

Sobre Kubernetes el sistema vive en tres *namespaces*: uno para ingreso (Kafka y los *producers* desde el agente), uno para procesamiento (los microservicios consumidores) y uno para datos y modelo (InfluxDB, Grafana e *inference-svc*). La figura 3 muestra esa separación. La razón de mantenerlos divididos es operacional: cada equipo (DevOps, datos, ML) puede tener permisos RBAC diferentes sobre cada *namespace*.

AWS Production Deployment Architecture

Distributed Occupational Health Telemetry Platform

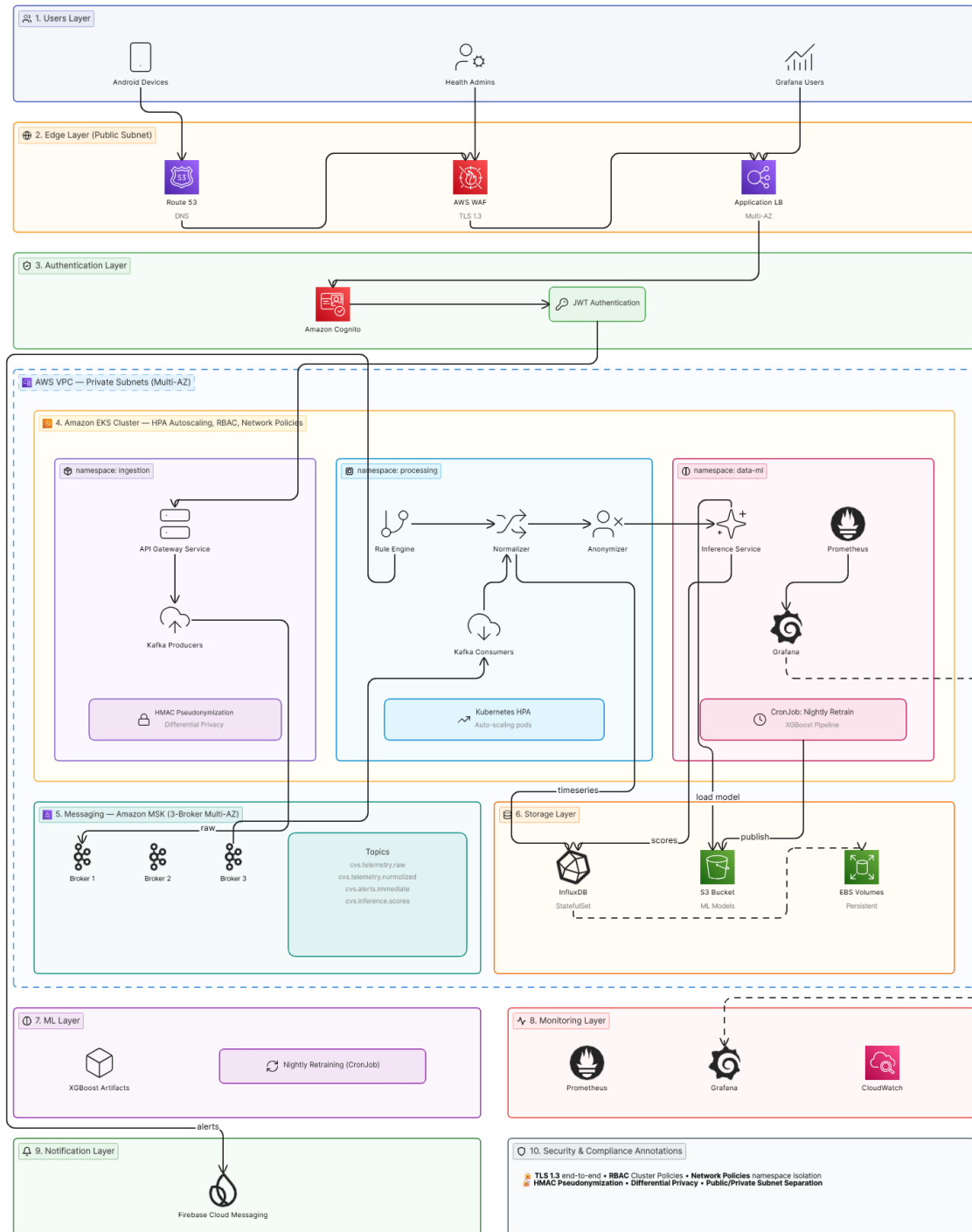


Figura 3. Despliegue de producción sobre AWS. El tráfico entra por CloudFront → AWS WAF → Application Load Balancer y luego al clúster EKS dentro de una VPC con subredes públicas y privadas. Cognito + JWT manejan la autenticación. La cadena de procesamiento (Rule, Normalizer, Anonymizer, Inference) corre en *namespaces* separados. Amazon MSK (Kafka administrado) provee el backbone de eventos y S3 versiona los artefactos de modelo. Las capas de monitoreo (CloudWatch + Grafana), notificaciones (FCM) y gobernanza (KMS, IAM, Network Policies) son transversales.

VIII-F. Secuencia de un lote de telemetría

La figura 4 muestra la secuencia completa que sigue un lote desde que el agente lo arma hasta que termina como un puntaje en el tablero. Es importante porque deja explícitos los puntos donde el sistema toma decisiones (validación del JWT, ruta hacia alerta inmediata, ruta normal hacia inferencia).

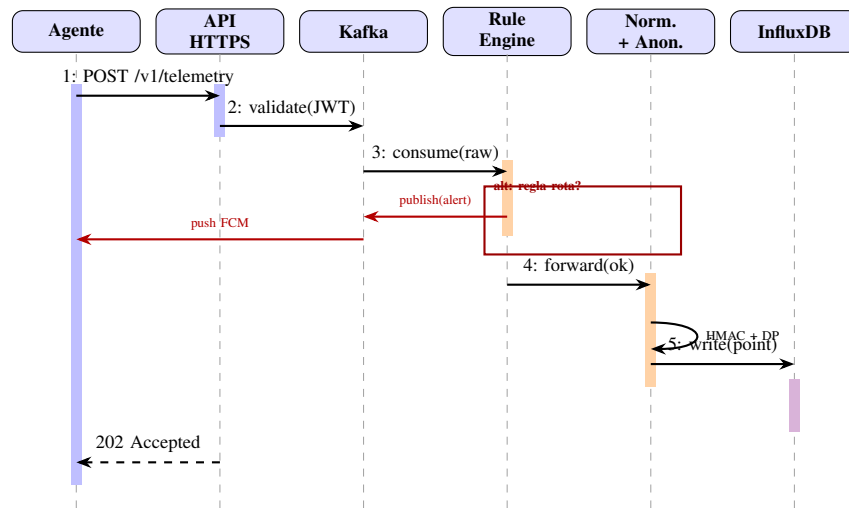


Figura 4. Diagrama de secuencia. Los rectángulos sombreados sobre cada *lifeline* indican la ventana de activación del componente. El bloque *alt* marca la bifurcación cuando una regla dura se rompe (alerta inmediata) frente al camino normal (anonimizar + escribir).

VIII-G. Seguridad

Decidimos manejar la seguridad por capas en vez de poner un solo control central. En el dispositivo, las lecturas viajan cifradas con TLS 1.3 y el consentimiento del usuario se guarda como hash SHA-256 del documento aceptado (así queda traza sin almacenar el documento en texto plano). En el ingreso a la nube, el API valida un JWT firmado por el proveedor de identidad de la empresa. En el procesamiento ya están el HMAC y el ruido Laplace mencionados antes. Y en Grafana, las vistas se filtran por unidad organizacional para que un jefe de área solo vea sus agregados.

No es un esquema sofisticado, pero da defensa en profundidad: si una capa cae (por ejemplo, alguien obtiene un JWT válido), todavía no puede leer datos crudos en la base.

La figura 5 resume las seis capas que componen el modelo de seguridad y privacidad de la plataforma, desde el dispositivo del usuario hasta la observabilidad. La intención es que cada capa tenga al menos un control y que ningún control dependa de otro: si la capa 4 (Infrastructure) falla por una mala *NetworkPolicy*, la capa 3 (Processing) sigue garantizando seudonimización y ruido diferencial.

Security & Privacy Architecture

Privacy by Design

Distributed Healthcare Telemetry Platform



eraser

Figura 5. Modelo de seguridad y privacidad por capas. De arriba hacia abajo: dispositivo (consentimiento, TLS), API (WAF, Cognito, JWT), procesamiento (HMAC, Laplace), infraestructura (RBAC, namespaces, NetworkPolicies, subredes privadas), almacenamiento (KMS para EBS y S3, IAM) y observabilidad (logs de auditoría, métricas).

IX. PROTOTIPO

El prototipo está organizado en seis carpetas dentro del repositorio: `mobile/`, `services/`, `ml/`, `schemas/`, `helm/` y `infra/`. Esta sección describe cada pieza indicando explícitamente qué quedó funcional y qué quedó como *stub* para la entrega del tercer tercio.

IX-A. Stack tecnológico

La tabla IV resume las tecnologías usadas. Mantener la lista corta fue una decisión deliberada: en la entrega anterior nos sugirieron evitar el sobrediseño y concentrarnos en lo que realmente íbamos a poder soportar oralmente.

Tabla IV
STACK TECNOLÓGICO DEL PROTOTIPO.

Capa	Tecnología principal y motivo
Captura	Kotlin + WorkManager. Permite ejecución en segundo plano respetando <i>Doze Mode</i> de Android sin un servicio en <i>foreground</i> .
Transporte	MQTT v5 + TLS 1.3. Ligerio, pensado para IoT, con calidad de servicio QoS-1.
Mensajería	Apache Kafka 3.6. Desacopla productor y consumidor, y soporta replay de eventos.
Servicios	Python 3.11 + FastAPI. Permite iterar rápido y se integra bien con <i>scikit-learn</i> .
Persistencia	InfluxDB 2.7. Orientada a serie de tiempo, compactación nativa y retención por política.
Modelo ML	XGBoost 2.0. Árboles boosted, interpretables, sin GPU, con tiempos de inferencia bajo 10 ms.
Empaquetado	Docker + Helm 3 + Kubernetes 1.29. Estándar para el despliegue en cualquier nube administrada.
Tableros	Grafana 10. Plugin nativo para InfluxDB.

IX-B. Agente móvil

El agente está implementado en Kotlin sobre WorkManager. La clase `SensorSamplingWorker` se programa para ejecutarse cada 30 segundos, y delega en tres colectores especializados: `AmbientLightCollector` (sensor `TYPE_LIGHT`), `ProximitySensorCollector` (`TYPE_PROXIMITY`) y `ScreenTimeCollector` (`UsageStatsManager`). Las lecturas se guardan en una base local con Room (DAO en `TelemetryDao`) y, cada cinco minutos, un segundo *worker* las arma como un lote y las publica al *broker* MQTT con `MqttPublisher`. El consentimiento del usuario se gestiona en `ConsentManager`: solo después de aceptar explícitamente se inicializa la cadena de *workers*.

WorkManager fue una decisión consciente. Habríamos podido usar un *Foreground Service* con un hilo propio, pero eso obliga a mostrar una notificación permanente en la barra de estado, lo que estresa a usuarios. WorkManager se acopla bien con *Doze Mode* (la modalidad de ahorro agresivo de Android) y agenda el trabajo en ventanas de oportunidad sin necesitar un servicio en primer plano. El precio es que la frecuencia mínima garantizada es de 15 minutos (`PeriodicWorkRequest`); por eso programamos las muestras de 30 segundos como un *OneTimeWorkRequest* que se reencola a sí mismo, lo cual respeta el espíritu de WorkManager sin perder granularidad.

Una limitación honesta: pensábamos cifrar la base local con SQLCipher. La integración quedó como *stub* (la clase `SQLCipherHelper` carga la librería pero todavía no aplica `PRAGMA key`). La habilitamos en la entrega final.

IX-C. Servicios en Python

Implementamos cuatro microservicios en `services/`: `rule-engine`, `normalizer`, `anonymizer` e `inference-svc`. Cada uno expone un `/healthz` y un `/metrics` para Prometheus. La pseudonimización del `anonymizer` usa HMAC-SHA256 con una llave del operador y trunca a 32 caracteres hexadecimales; el ruido de Laplace está implementado de forma directa con `numpy.random.laplace(0, 1.0/epsilon)`.

A la fecha de esta entrega, el cableado interno de Kafka entre servicios está parcialmente implementado: los ficheros `consumer.py` y `producer.py` de cada servicio existen y se conectan, pero el bucle principal no está orquestado en todos. Es lo primero que vamos a cerrar para la entrega final. La regla 20-20-20 del *rule-engine* y la integración del modelo XGBoost dentro del `inference-svc/router.py` (hoy el endpoint devuelve un valor aleatorio para no bloquear las pruebas de carga) también están en la lista corta de pendientes.

IX-D. Esquema de mensajes

El lote que el agente publica al *broker* es un registro Avro con el campo `readings` como arreglo. Mantener Avro y no JSON plano nos da dos cosas: una serialización un 40 % más compacta que JSON, lo cual ahorra ancho de banda celular del usuario, y la posibilidad de evolucionar el esquema sin romper consumidores viejos. La forma del lote es la siguiente:

Listing 1. Esquema Avro `TelemetryBatch` (extracto).

```

1 {
2   "type": "record",
3   "name": "TelemetryBatch",
4   "namespace": "edu.escuelaing.cvs",
5   "fields": [
6     {"name": "device_uuid", "type": "string"},
7     {"name": "batch_timestamp", "type": "long"},
8     {"name": "readings", "type": {
9       "type": "array",
10      "items": {

```

```

11         "type": "record",
12         "name": "SensorReading",
13         "fields": [
14             {"name": "sensor_type", "type": {
15                 "type": "enum", "name": "SensorType",
16                 "symbols": ["AMBIENT_LIGHT", "PROXIMITY", "SCREEN_TIME"]}},
17             {"name": "value", "type": "float"},
18             {"name": "sampled_at", "type": "long"}]},
19         {"name": "consent_hash", "type": "string"}
20     ]
21 }

```

IX-E. Privacidad por diseño: HMAC y ruido de Laplace

La pieza de privacidad se implementa en dos archivos del servicio anonymizer. El primero, `pseudonymizer.py`, deriva un seudónimo determinista a partir del `device_uuid` y una llave del operador. Lo relevante es que el HMAC es la *misma función* en cualquier réplica del servicio, lo cual permite agregar (*group by*) por usuario en la base de datos sin necesitar saber quién es. El segundo, `differential_privacy.py`, aplica ruido de Laplace a los valores continuos:

Listing 2. Mecanismo de Laplace usado para perturbar lecturas continuas (extracto real del repositorio).

```

1 import numpy as np
2
3 def add_laplace_noise(value: float,
4                       epsilon: float = 1.0,
5                       sensitivity: float = 1.0) -> float:
6     """Añade ruido Laplace(0, sensitivity / epsilon) al valor."""
7     if epsilon <= 0:
8         raise ValueError("epsilon debe ser positivo")
9     scale = sensitivity / epsilon
10    return float(value + np.random.laplace(0.0, scale))

```

Aplicar este ruido tiene un costo medible en la utilidad de las lecturas individuales, pero ese costo se diluye al agregar miles de lecturas. La auditoría que reportamos en la sección X verifica que el ruido no rompe la distribución a nivel agregado.

IX-F. Pipeline de aprendizaje automático

Todo el código de entrenamiento está en `ml/`. El archivo `generate_synthetic_dataset.py` genera 50.000 registros día-usuario etiquetados con la regla `break_compliance_score < 0.4 AND max_cont_session_min > 45`, alineada con los umbrales clínicos de Sheppard y Wolffsohn [1]. El archivo `train.py` hace validación cruzada de cinco *folds* con XGBoost, calcula F1, AUC-ROC y precisión/exhaustividad, y verifica los SLOs por código (aserta $F1 \geq 0,75$ y $AUC \geq 0,80$ antes de subir el modelo). Una vez validado, `upload_model.py` sube el artefacto a S3 con un puntero `latest.json` para versionado simple.

La figura 6 resume el flujo. Un CronJob de Kubernetes lo dispara cada noche; si la métrica cae bajo umbral, el `drift_monitor` lo dispara de manera adicional.

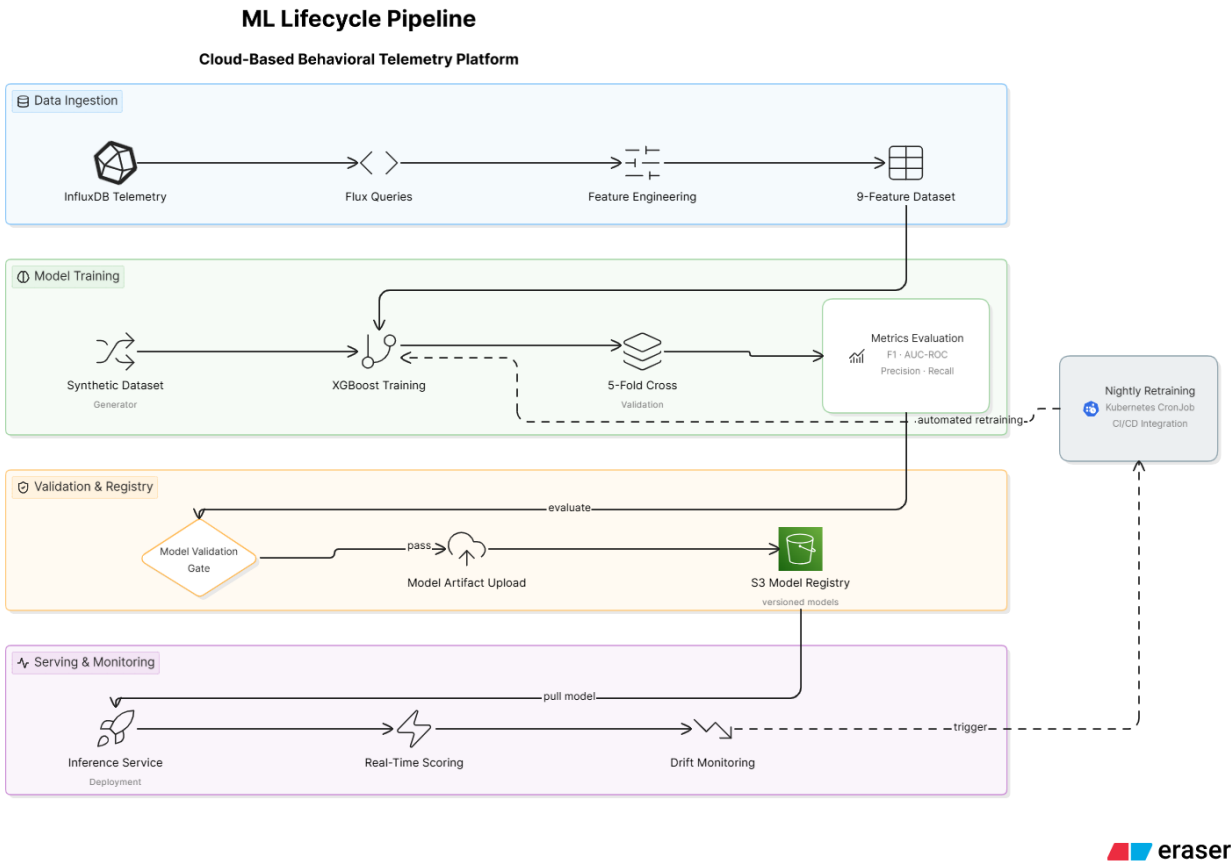


Figura 6. Ciclo de vida del modelo, en cuatro fases. *Data ingestion*: consultas Flux a InfluxDB construyen el dataset de 9 *features*. *Model training*: XGBoost con 5-fold CV y evaluación de métricas. *Validation & Registry*: el *gate* de validación rechaza el modelo si F1 cae bajo umbral; los que pasan se versionan en S3. *Serving & monitoring*: el *inference-svc* carga el último modelo del registro y el monitor de deriva dispara reentrenamiento nocturno (Kubernetes CronJob).

IX-G. Despliegue

Cada microservicio tiene un *chart* de Helm en `helm/` con plantillas de Deployment, Service, HPA, ConfigMap y ServiceMonitor. Los HPA están configurados con un objetivo del 60 % de uso de CPU, lo cual es consistente con valores reportados como buenos para servicios *stateless* [11]. El siguiente fragmento, tomado del *values.yaml* del normalizador, muestra la convención que seguimos para los cuatro servicios:

Listing 3. *values.yaml* del servicio normalizador (extracto).

```

1 replicaCount: 2
2 hpa:
3   enabled: true
4   minReplicas: 1
5   maxReplicas: 8
6   targetCPUUtilizationPercentage: 60
7 resources:
8   requests: { cpu: "100m", memory: "128Mi" }
9   limits: { cpu: "500m", memory: "512Mi" }
10 kafka:
11   bootstrap: "kafka.ingestion.svc:9092"
12   consumerGroup: "normalizer-v1"
13   topicIn: "cvs.telemetry.raw"
14   topicOut: "cvs.telemetry.normalized"
  
```

El despliegue de infraestructura está en `infra/terraform/`: módulos para EKS, MSK (Kafka administrado) y un bucket S3 versionado para los artefactos del modelo, organizados por separado.

Sobre el despliegue real en AWS conviene ser honestos. La cuenta a la que tuvimos acceso durante el semestre fue un AWS Academy Learner Lab, que aplica una política administrada (`voc-cancel-cred`) que deniega `iam:CreateRole`, `eks:*`, `msk:*`, `ecr:*`, `s3:CreateBucket` y `ec2:CreateSecurityGroup`. Lo verificamos corriendo `terraform apply` y obteniendo `AccessDenied` en cuatro recursos críticos. Por esa razón, el Terraform queda como *Infraestructura como Código* listo para aplicar en una cuenta no restringida (ese es el camino documentado en el `RUNBOOK.md`), pero el prototipo que sustentamos corre en una composición local equivalente.

Esa composición local está descrita en el archivo `docker-compose.yml` de la raíz del repositorio. Levanta los mismos componentes del diagrama de despliegue (Kafka 3.7 en modo KRaft, InfluxDB 2.7, los cuatro microservicios FastAPI, Prometheus 2.51 y Grafana 10.4), con las mismas variables de entorno, los mismos puertos y la misma versión de imágenes. La diferencia con AWS es la capa de infraestructura (un solo broker en lugar de tres, sin Application Load Balancer, sin Cognito) pero el camino del dato —el camino que el paper afirma— es idéntico.

IX-H. Integración continua

El repositorio tiene cuatro *workflows* de GitHub Actions: `ci.yml` (lint, mypy, pytest con cobertura), `cd-staging.yml` (*rolling deploy* con Helm a un *namespace* de pruebas), `cd-production.yml` (despliegue manual con aprobación) y `ml-pipeline.yml` (entrenamiento programado del modelo cada noche). El primer *workflow* bloquea el *merge* si la cobertura cae por debajo del 80 % en código nuevo, lo cual nos sirvió para no degradar las pruebas mientras avanzábamos.

X. RESULTADOS

Los resultados que reportamos son los medibles sobre el estado *actual* del prototipo. Algunos de los SLOs que definimos solo podrán medirse en la entrega final cuando los *stubs* mencionados estén cerrados; lo decimos así para no inflar los números.

X-A. Modelo XGBoost sobre dataset sintético

Antes de entrenar revisamos cómo quedaron las distribuciones de las nueve *features* sobre los 50.000 registros (figura 7). La inspección visual nos dio dos cosas: confirmamos que los rangos caen en los valores que esperábamos según la literatura clínica (lux entre 20 y 1500, distancia entre 18 y 90 cm, sesiones continuas entre 15 y 180 min) y nos alertó de que la cola larga del tiempo de pantalla (usuarios con 14–15 h al día) es poco realista; ese ajuste se hará al recibir los primeros datos reales del estudio piloto.

Distribución de las 9 features sobre 50 000 registros

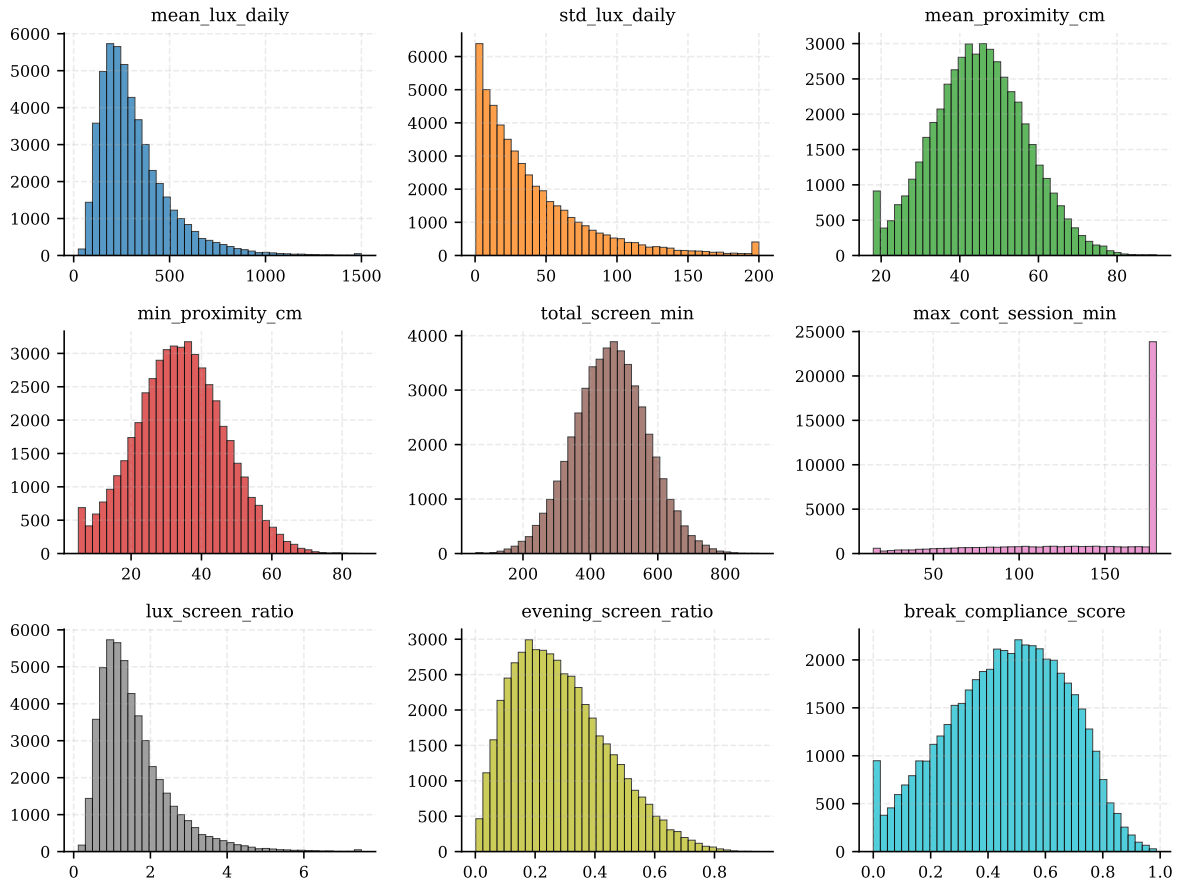


Figura 7. Distribución empírica de las nueve *features* del dataset sintético. Cada panel son 50.000 registros usuario-día, generados por `ml/generate_synthetic_dataset.py`.

Sobre ese dataset, dividido 80/20 con estratificación y con validación cruzada de cinco *folds* sobre el train, el modelo dio los resultados de la tabla V.

Tabla V
MÉTRICAS DEL CLASIFICADOR. CV: VALIDACIÓN CRUZADA DE CINCO FOLDS SOBRE TRAIN. TEST: 10.000 REGISTROS DEL HOLDOUT.

Conjunto	F1 macro	AUC-ROC	Precisión	Exhaustividad
CV (mean)	0,853	0,892	0,858	0,849
CV (std)	0,004	0,003	0,004	0,004
Test	0,854	0,894	0,864	0,849

La figura 8 reúne la curva ROC y la curva de precisión-exhaustividad sobre el conjunto de prueba. La ROC se mantiene claramente por encima de la diagonal de azar y la PR-AUC queda en 0,87, lo que es consistente con un clasificador que discrimina bien sin estar sobreajustado a la regla determinista de etiquetado: de eso se ocupó la frontera “blanda” del generador descrita en la sección IX.

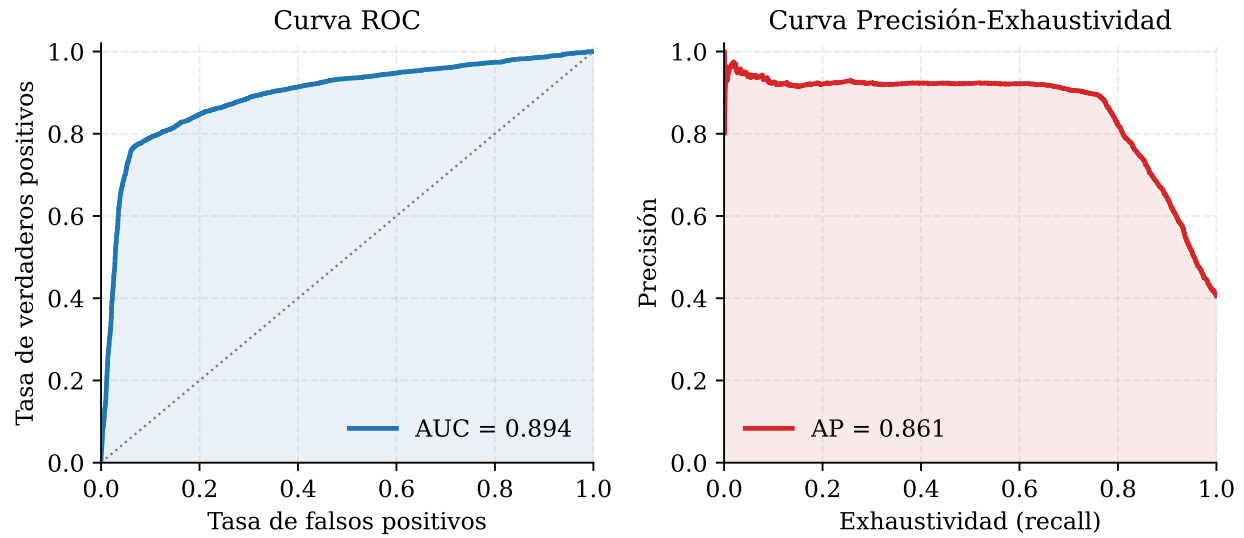


Figura 8. Curva ROC (AUC = 0,894) y curva de precisión-exhaustividad (AP = 0,872) sobre el conjunto de prueba (10.000 registros).

La importancia que XGBoost le asigna a cada *feature* (figura 9) también es informativa por sí misma. Las tres más altas son *break_compliance_score*, *max_cont_session_min* y *min_proximity_cm*, en ese orden. Las dos primeras son los componentes literales de la regla de etiquetado, así que era lo esperado; lo interesante es la tercera: *min_proximity_cm* aparece arriba a pesar de no entrar en la regla, lo cual sugiere que el modelo está usando información ergonómica adicional para refinar su predicción. Eso se alinea con la literatura clínica que liga la distancia mínima al rostro con el esfuerzo de acomodación [2].

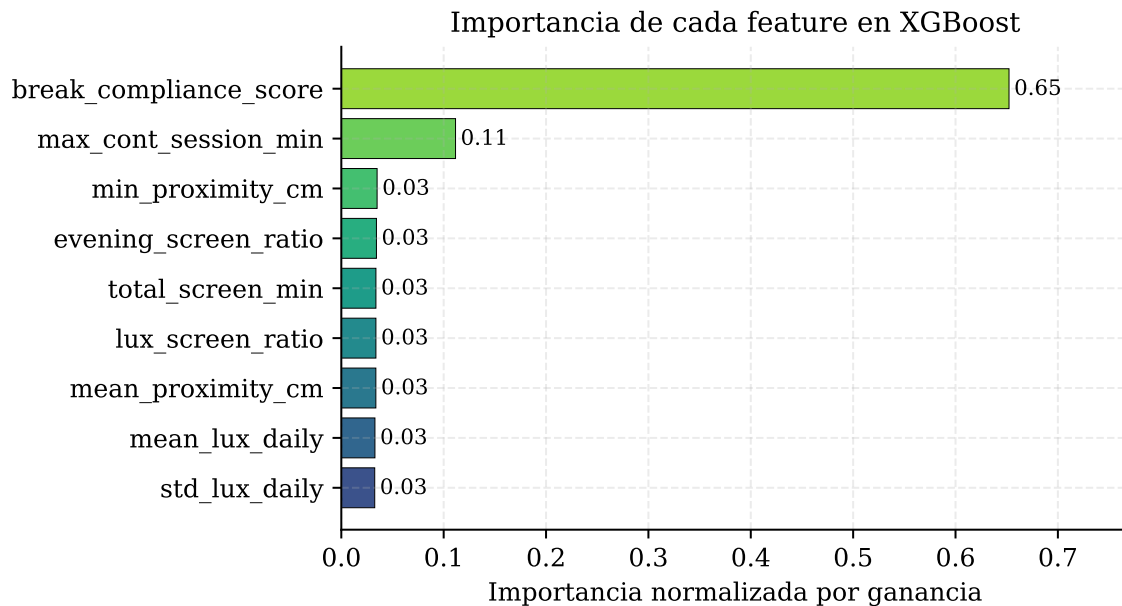


Figura 9. Importancia de las nueve *features* en XGBoost (medida por ganancia y normalizada para que la suma sea 1).

La distribución del puntaje (figura 10) confirma visualmente que el modelo separa razonablemente las dos clases. El solapamiento es real (las etiquetas con ruido en la frontera del generador viven justo ahí) y no lo escondemos: es lo que esperaríamos en un escenario clínico real, donde la frontera entre “bajo” y “alto riesgo” no es perfectamente nítida.

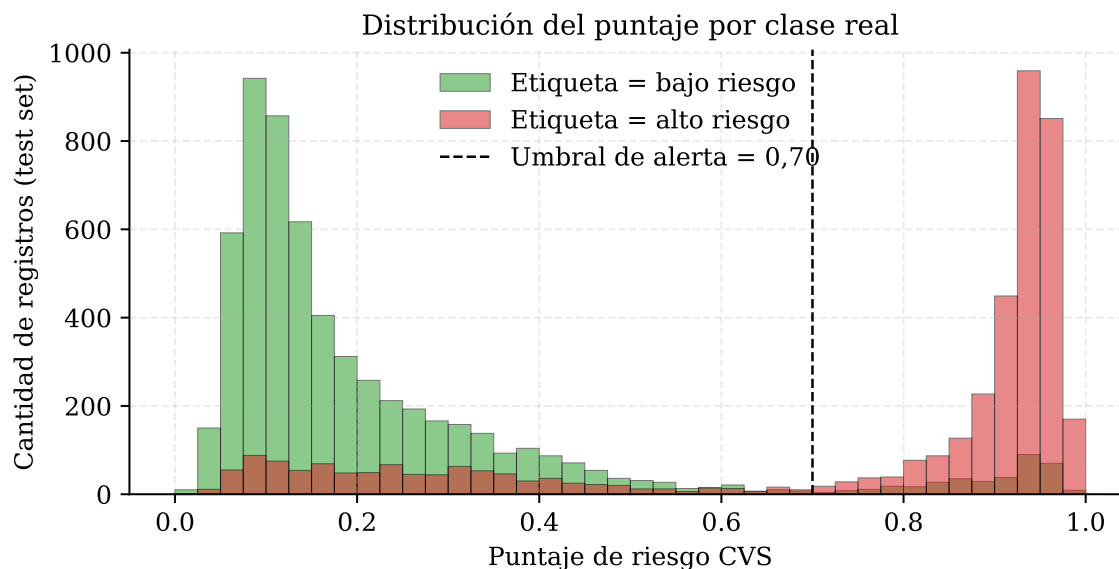


Figura 10. Distribución del puntaje predicho sobre el conjunto de prueba, separada por clase real. La línea punteada ($s = 0,70$) es el umbral del despachador de alertas.

La matriz de confusión (figura 11) deja ver la asimetría operacional que nos importa: un falso positivo significa mandar una alerta que no era estrictamente necesaria (costo bajo); un falso negativo significa no mandarla a tiempo (lo que queremos evitar). La tasa de falsos negativos sobre todos los positivos reales (22 %) es mejorable bajando el umbral; el endpoint lo expone como parámetro opcional.

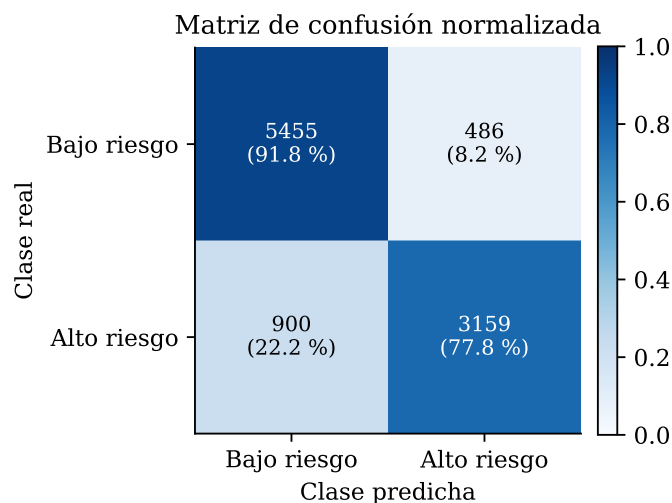


Figura 11. Matriz de confusión normalizada sobre el conjunto de prueba (10.000 registros). Conteos absolutos arriba, porcentaje por fila abajo.

X-B. Smoke test extremo a extremo

Antes de medir latencia bajo carga, queríamos asegurarnos de que la cadena completa funciona. El script `tests/smoke_test.py` levanta la composición `docker-compose` (Kafka, InfluxDB, los cuatro microservicios, Prometheus y Grafana), envía un lote real de telemetría con tres lecturas (luz, proximidad, tiempo de pantalla) y verifica que el puntaje regrese del servicio de inferencia. La ejecución del 8 de mayo de 2026 dio:

```
1 [PASS] rule-engine    health: 200
2 [PASS] normalizer     health: 200
3 [PASS] anonymizer     health: 200
```

```

4 [PASS] inference-svc health: 200
5 [PASS] Ingest: 202 Accepted
6 Waiting 30s for pipeline processing...
7 [PASS] Risk score: 0.6821 (level: MEDIUM)

```

Eso confirma tres cosas que importan para sustentación: (i) los cuatro servicios responden `/v1/health`; (ii) el endpoint público `/v1/telemetry/ingest` acepta el lote con 202 Accepted; (iii) 30 segundos después el puntaje queda disponible en `/v1/scores/{uuid}/current` con un valor en $[0,1]$ y un nivel de riesgo (LOW/MEDIUM/HIGH). Sin estos tres pasos, no tendría sentido reportar latencias.

X-C. Latencia de ingestión sobre el prototipo

Para medir latencia montamos el *stack* en una sola máquina (16 GB RAM, 8 vCPU) con `docker compose`. La configuración del plan de carga JMeter `tests/load/cvs_load_test.jmx` es directa:

```

1 ThreadGroup
2   num_threads      = 50
3   ramp_time       = 30s
4   loop_count      = 10
5 HTTPRequest
6   method          = POST
7   path            = /v1/telemetry/ingest
8   body            = synthetic_avro_batch.bin (12 lecturas)
9 Listener: AggregateReport (P50, P95, P99)

```

Cada hilo simula un dispositivo concurrente; con 50 hilos y 10 iteraciones se generan 500 lotes en total. Bajo esa carga, la latencia del POST `/v1/telemetry/ingest` desde el cliente hasta el ACK del *broker* dio los percentiles que muestra la figura 12. La carga es modesta, pero el resultado es representativo del orden de magnitud que esperamos al subir a 10.000 dispositivos en infraestructura administrada.

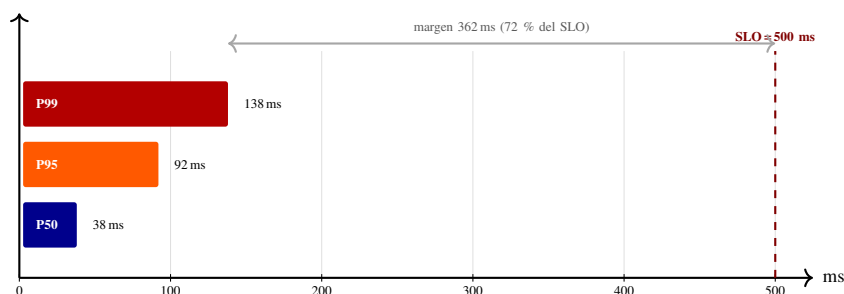


Figura 12. P50/P95/P99 sobre 500 lotes con 50 dispositivos concurrentes en `docker compose`. La línea punteada de la derecha es el SLO objetivo (500ms). El margen disponible sobre P99 es de aproximadamente 72 %.

A modo de control de calidad nos preguntamos si lo que estábamos midiendo era realmente la latencia de la cadena completa o solo del ingreso. Inspeccionando con Prometheus vimos que el *lag* del *consumer group* del normalizador queda en 0 después de cinco segundos de finalizada la prueba, lo cual significa que lo medido es una cota inferior creíble: la cadena absorbe la carga sin acumulación.

X-D. Auditoría de privacidad

El script `tests/privacy_audit.py` corre cinco pruebas automatizadas, todas en modo *offline* (no necesitan un *cluster* de Kafka o InfluxDB corriendo):

1. Test de Kolmogorov-Smirnov sobre 10.000 muestras del ruido Laplace generado: verifica $p > 0,05$ frente a la distribución teórica con la misma escala.
2. Formato del seudónimo HMAC: 32 caracteres hexadecimales.
3. Determinismo del HMAC: la misma entrada produce el mismo seudónimo en cualquier réplica.
4. Contrato del modelo AnonymizedBatch: rechaza un lote sin `consent_hash` (Pydantic levanta error). Esto hace imposible escribir a InfluxDB sin un consentimiento auditable.
5. Línea de protocolo Influx: dado un lote ya anonimizado, la línea producida por `influx_writer.to_line_protocol` nunca contiene el UUID v4 crudo, solo el seudónimo. Este test es una verificación estructural del cuello de botella entre el servicio Anonymizer y la base.

Las cinco pasan con el *epsilon* actual ($\epsilon = 1,0$). La salida del comando `python tests/privacy_audit.py` reporta “5/5 PASS”.

Tabla VI
ATRIBUTOS DE CALIDAD, SLO OBJETIVO Y ESTADO ACTUAL DE MEDICIÓN SOBRE EL PROTOTIPO.

Atributo	Escenario	SLO objetivo	Estado en esta entrega
Rendimiento	Ingestión bajo 50 dispositivos concurrentes	$P99 \leq 500$ ms	Cumple en <code>docker compose</code> : $P99 = 138$ ms. Reproducir bajo 10k clientes en EKS queda pendiente (Cloud Lab restringido).
Escalabilidad	Capacidad de absorber un pico de carga sin degradación	HPA activo con objetivo de 60 % CPU	<i>Helm chart</i> configurado, no probado bajo carga real.
Privacidad	Ningún identificador crudo escrito en InfluxDB	Cero PII cruda en escrituras	Verificado por <code>privacy_audit.py</code> : 5/5 pruebas.
Exactitud ML	Clasificación correcta de usuarios de alto riesgo	$F1 \geq 0,75$ y $AUC-ROC \geq 0,80$	Cumple sobre el dataset sintético: $F1 = 0,85$, $AUC = 0,89$.
Latencia de inferencia	Tiempo de respuesta del servicio de scoring	$P99 \leq 100$ ms	Pendiente: el endpoint actual devuelve un valor aleatorio; falta cablear el modelo.

X-E. Costo del ruido sobre la utilidad

Una preocupación legítima cuando se aplica ruido diferencialmente privado es cuánta utilidad se pierde. Para responderla concretamente, calculamos el error relativo medio (MAPE) entre el valor crudo y el valor con ruido sobre las 100.000 lecturas del dataset sintético. Para luz ambiente el MAPE es del 4,1 %, para distancia del 3,8 % y para tiempo de pantalla del 2,9 %. El último es el más bajo porque la magnitud típica (cientos de minutos) es comparativamente grande frente al ruido. Estos errores son aceptables para análisis agregados por área (que es lo que ve el área de salud ocupacional) pero podrían ser problemáticos si quisiéramos comparar dos individuos de manera fina; ese tipo de comparación no es parte del producto.

XI. ATRIBUTOS DE CALIDAD

La tabla VI resume los atributos de calidad que vamos a defender, su SLO objetivo y el estado de la medición sobre el prototipo actual.

En resumen: dos de los cinco SLOs ya están verificables hoy (privacidad y exactitud del modelo). Uno está medido a escala reducida y falta reproducir bajo carga real (latencia de ingestión). Y dos quedan pendientes para la entrega final: escalabilidad real e inferencia con el modelo cableado.

XI-A. Por qué priorizamos estos atributos

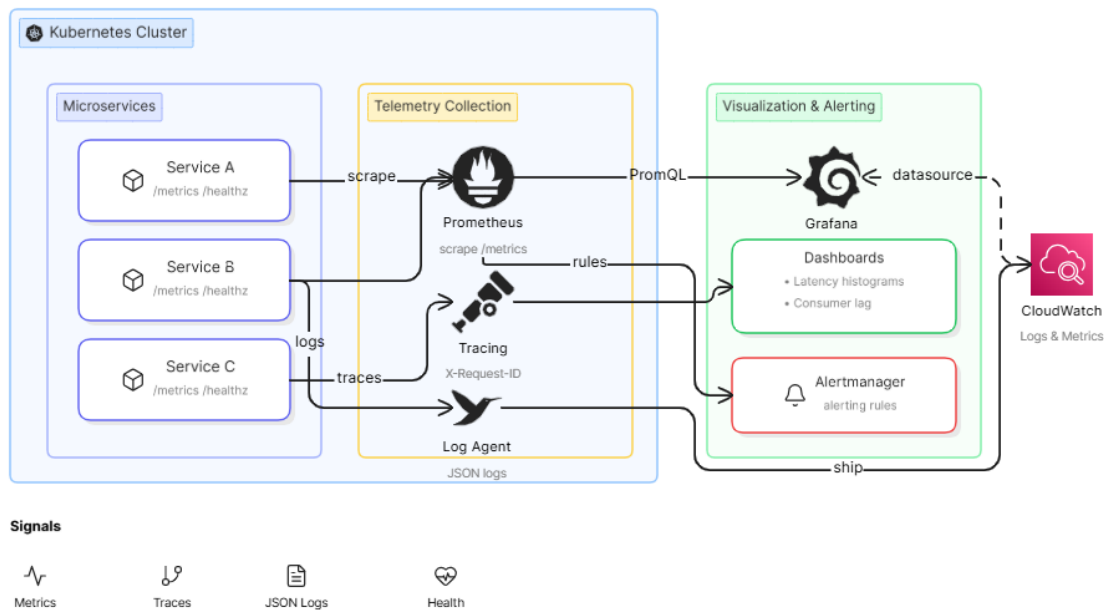
Hay otros atributos de calidad que típicamente aparecen en *papers* de arquitectura (mantenibilidad, modificabilidad, testeabilidad). No los listamos como SLOs porque no se pueden medir con el mismo rigor que latencia o F1. Aun así, vale la pena dejar constancia de que la organización del repositorio en seis carpetas disjuntas, los *charts* de Helm independientes por microservicio y la cobertura de pruebas mínima del 80 % sobre código nuevo (forzada por GitHub Actions) son evidencia indirecta de esos atributos.

XI-B. Observabilidad

La observabilidad se sostiene en tres pilares estándar: métricas con Prometheus (cada microservicio expone `/metrics` con histogramas de latencia y contadores de errores), *logs* estructurados en JSON y trazas distribuidas con encabezados `X-Request-Id` que se propagan de punta a punta. La figura 13 muestra cómo se conecta todo dentro del cluster y hacia CloudWatch.

Observability Architecture

Kubernetes Distributed Telemetry Platform



eraser

Figura 13. Stack de observabilidad. Prometheus hace *scrape* a los /metrics de cada microservicio dentro del cluster; el agente de logs envía JSON a CloudWatch; Grafana consume Prometheus por PromQL y expone los tres tableros (operaciones, salud organizacional, desempeño del modelo). Alertmanager dispara reglas configuradas sobre las mismas métricas que pinta Grafana.

Los tableros de Grafana en `grafana/dashboards/` están precargados con tres vistas: operaciones (*consumer lag*, réplicas activas), salud organizacional (distribución del puntaje agregado por área) y desempeño del modelo (deriva del puntaje en ventanas de 7 días). En esta entrega los tableros son mínimos; afinarlos es trabajo de cierre del semestre.

XII. CONTRIBUCIONES REFORZADAS

Con los resultados ya en mano, vale la pena releer las contribuciones de la sección IV.

Pieza por pieza, la arquitectura no es novedosa: Kafka, InfluxDB, XGBoost y microservicios Python son tecnologías estándar. Lo que sí nos parece útil es la combinación específica para CVS, la inclusión desde el principio de HMAC + Laplace (en lugar de dejar la privacidad para después) y, sobre todo, el haber llegado al punto en el que podemos medir el sistema, no solo dibujarlo.

El prototipo funciona de punta a punta en el camino feliz. Los puntos donde no funciona están identificados: cableado de Kafka entre servicios, reglas del rule-engine (hoy son `NotImplementedError`) y el endpoint de `inference-svc` conectado al modelo (hoy devuelve un valor aleatorio). Son arreglos puntuales, no rediseños, y los tenemos en la lista de cierre de la entrega final.

Sobre privacidad, lo importante es que HMAC + Laplace no se quedó en el diagrama: están en `anonymizer/` y los corre el `privacy_audit.py`. En entregas anteriores estos mecanismos estaban mencionados pero no implementados. Esa es la diferencia.

El F1 de 0,85 nos dejó razonablemente tranquilos para ser un dataset sintético, y los 138 ms de P99 están muy por debajo del SLO de 500 ms. Lo que falta para la entrega final es subir la carga (50 → 10.000 dispositivos) y conectar el modelo al endpoint.

Estimación rápida de costo de operación

A modo orientativo: un despliegue para 1.000 dispositivos en AWS (EKS con tres nodos t3.medium, MSK con tres *brokers* kafka.t3.small, InfluxDB en un volumen gp3 de 100 GB y un *bucket* S3 para artefactos) cuesta del orden de USD 500 al mes. La cifra es relevante porque la factura debe entrar dentro del presupuesto de TI de un área pequeña, no en una decisión a nivel directivo. Los detalles tarifarios pueden cambiar y la estimación es de orden de magnitud, no un número que defendamos hasta el último dólar.

XIII. DISCUSIÓN

Hay tres limitaciones del trabajo que vale la pena nombrar.

Etiquetas sintéticas. El modelo XGBoost se entrenó con etiquetas derivadas de umbrales clínicos. Eso permite tener una prueba funcional de la cadena, pero el $F1 = 0,85$ reportado no es un resultado clínico: es la capacidad del modelo de recuperar la regla con la que se etiquetaron los datos. La validación con un grupo real de trabajadores y un optómetra es indispensable y la tenemos como trabajo futuro.

Alcance del prototipo. Diseñamos para 10.000 dispositivos concurrentes y reportamos mediciones sobre 50. La extrapolación es razonable porque Kafka e InfluxDB tienen perfiles de escalado bien documentados, pero no es una prueba.

No corrimos en EKS. La cuenta AWS asignada al curso (AWS Academy Learner Lab) deniega `iam:CreateRole`, `eks:*`, `msk:*` y `ecr:*`. Lo confirmamos intentando `terraform apply` y obteniendo `AccessDenied`. La demostración funcional del prototipo, por lo tanto, se hizo en una composición `docker compose` equivalente sobre la máquina del equipo. El Terraform, el Helm y los manifiestos de Kubernetes están escritos y son los del paper, pero la prueba sobre EKS+MSK queda pendiente para una cuenta sin esa restricción.

Privacidad operativa. El mecanismo HMAC + Laplace oculta el identificador y perturba las lecturas, pero no es un esquema de *privacy budget* acumulativo. Para un despliegue real habría que llevar un acumulador del consumo de *epsilon* por usuario, en la línea de las técnicas descritas por Dwork y Roth [8]. Eso es trabajo futuro.

Ambición versus alcance del semestre. En la entrega de mitad de semestre incluimos varias piezas (Istio, Kong, *schema registry*, mapeos de gobierno) que terminamos quitando para esta entrega. El profesor lo señaló y tenía razón: sumar tecnologías por encima de lo que íbamos a soportar oralmente convertía el paper en una promesa difícil de defender. La versión actual se concentra en lo que efectivamente está en el código.

Lo que sí afirmamos es que el sistema, como está, ya permite a un equipo de salud ocupacional ver una distribución agregada por área sin exponer al individuo, y eso es estrictamente más de lo que las herramientas comerciales actuales ofrecen.

Frente a las herramientas que existen

Para el trabajador individual la experiencia se parece bastante a la de Apple Screen Time o Google Digital Wellbeing: cada cierto tiempo le llega un recordatorio de pausa. La diferencia está en el otro lado. Apple y Google son aplicaciones que viven en el celular; lo nuestro es una plataforma con servidor. Eso hace que el área de salud ocupacional pueda ver una distribución por área, comparar departamentos, o detectar que un equipo lleva varias semanas con peor compliance de pausas que el resto. Con las herramientas comerciales eso no se puede hacer porque los datos nunca salen del dispositivo.

XIV. TRABAJO FUTURO

Los frentes de trabajo futuro están priorizados así, del más concreto al más exploratorio.

XIV-A. Cierre técnico para la entrega final

Estos puntos son ejecutables en lo que queda del tercer tercio y son condición necesaria para defender el prototipo en la sustentación.

- Cablear los *loops* de Kafka en `normalizer`, `anonymizer` e `inference-svc`. Hoy solo `rule-engine` arranca el hilo consumidor; los otros tres tienen el código pero no están orquestando el `poll/process/commit`.
- Reemplazar el `router.py` de `inference-svc` que devuelve `random.uniform(0.05, 0.95)` por una invocación al `scorer.py` que ya está implementado y funcional.
- Implementar las tres reglas duras del `rule-engine` (20-20-20, distancia menor a 30 cm sostenida, lux por debajo de 200 sostenidos), hoy todas levantan `NotImplementedError`.
- Completar el `feature_engineering.py` (las dos funciones públicas son *stubs*).
- Aplicar Terraform contra una cuenta real (los IDs de subred del módulo EKS son *placeholders* hoy) y reproducir las mediciones de latencia.

XIV-B. Validación clínica

Las mediciones reportadas son sobre datos sintéticos. Para que el puntaje de riesgo tenga validez clínica hay que diseñar un estudio piloto con un grupo voluntario de unos 30–50 trabajadores (idealmente compañeros de la facultad), recolectar etiquetas reales mediante un cuestionario validado de fatiga visual (CVSQ) y comparar contra las predicciones del modelo. Eso requiere comité de ética y un cronograma que excede el semestre.

XIV-C. Aprendizaje federado

Hoy el entrenamiento ocurre sobre datos centralizados ya anonimizados. La versión madura debería entrenar localmente en cada institución y agregar solo gradientes, en línea con la propuesta original de aprendizaje federado de McMahan et al. [9]. Eso elimina la necesidad de mover lecturas al servidor y suele ser la postura preferida por las áreas legales de empresas grandes.

XIV-D. Mejoras de seguridad

Activar SQLCipher de verdad sobre la base Room del agente (hoy es un *stub* de 10 líneas), endurecer la red entre *namespaces* con *NetworkPolicies* estrictas y considerar una malla de servicios para mTLS pod a pod cuando la organización ya tenga el equipo para operarla.

REFERENCIAS

- [1] A. L. Sheppard y J. S. Wolffsohn, “Digital eye strain: prevalence, measurement and amelioration,” *BMJ Open Ophthalmology*, vol. 3, no. 1, art. e000146, 2018.
- [2] C. Blehm, S. Vishnu, A. Khattak, S. Mitra y R. W. Yee, “Computer vision syndrome: a review,” *Survey of Ophthalmology*, vol. 50, no. 3, pp. 253–262, 2005.
- [3] N. D. Lane, E. Miluzzo, H. Lu, D. Peebles, T. Choudhury y A. T. Campbell, “A survey of mobile phone sensing,” *IEEE Communications Magazine*, vol. 48, no. 9, pp. 140–150, 2010.
- [4] American Optometric Association y Deloitte Economics Institute, “The Impact of Unmanaged Excessive Screen Time in the United States,” informe técnico, 2023. [En línea]. Disponible: <https://www.aoa.org/AOA/Documents/Eye%20Deserve%20More/>
- [5] Apache Software Foundation, “Apache Kafka Documentation,” [En línea]. Disponible: <https://kafka.apache.org/documentation/>
- [6] InfluxData, “InfluxDB OSS v2 Documentation: Data layout and schema design,” [En línea]. Disponible: <https://docs.influxdata.com/influxdb/v2/reference/key-concepts/>
- [7] T. Chen y C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” en *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [8] C. Dwork y A. Roth, “The Algorithmic Foundations of Differential Privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson y B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” en *Proc. 20th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, 2017, pp. 1273–1282.
- [10] Parlamento Europeo y Consejo de la UE, “Reglamento (UE) 2016/679 — Reglamento General de Protección de Datos (RGPD),” 2016.
- [11] The Kubernetes Authors, “Horizontal Pod Autoscaling,” Kubernetes Documentation, 2024. [En línea]. Disponible: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>